📘 سند رسمی فنی — نسخه ۲.۹.۰

برچسب‌خوان هوشمند دارو و محصولات LabelVoice

معماری، کارکرد، فناوری‌ها و ساختار شکست کار (WBS) پروژه — سامانهٔ دو زیرسیستمی دسکتاپ و موبایل که فقط «متن روی بسته» را با OCR می‌خواند، فیلدها را با شواهد قابل ردیابی استخراج می‌کند و پاسخ فارسی ایمن تولید می‌کند؛ کاملاً روی دستگاه، بدون ارسال هیچ داده‌ای.

👨‍💻 توسعه و طراحی: **حمید جلیلوند** و **نوید بدری**

۲	۲۳۸	۱۲	۴
زیرسیستم هم‌منطق دسکتاپ + موبایل	آزمون خودکار pytest + Node + E2E	بستهٔ کاری WBS نگاشت‌شده به فایل‌ها	موتور OCR به انتخاب کاربر

■ بخش ۱

نمای کلی پروژه

برچسب‌خوان هوشمند دارو و محصولات (LabelVoice) سامانه‌ای است که با گرفتن یک عکس از جعبهٔ دارو یا بسته‌بندی محصول، متن چاپ‌شده روی آن را با OCR استخراج می‌کند، فیلدهای کلیدی (نام، مادهٔ مؤثره، قدرت درج‌شده، شکل دارویی، تعداد، تاریخ انقضا، سری، سازنده) را با وضعیت پنج‌گانه و شواهد قابل ردیابی استخراج می‌کند و پاسخی به زبان فارسی تولید می‌کند که می‌تواند با خوانش صوتی پخش شود. خط قرمز طراحی از ابتدا صریح بوده است: سامانه فقط «بازخوانی آنچه روی بسته نوشته شده» است؛ قدرت درج‌شده هرگز «دوز مصرف» نامیده نمی‌شود، هیچ توصیهٔ پزشکی تولید نمی‌شود و هر فیلد استخراج‌شده باید به شناسهٔ قطعه‌های OCR (شواهد) متصل باشد.

پروژه دو زیرسیستم با منطق عیناً یکسان دارد: **دسکتاپ** با پایتون (Streamlit + OpenCV + Tesseract 5 و موتورهای اختیاری) و **وب/موبایل** با جاوااسکریپت خالص (PWA نصب‌شدنی + Tesseract.js روی WebAssembly + بستهٔ اندروید WebView). هر قاعدهٔ پردازشی دقیقاً دو بار در دو زبان پیاده شده و آزمون‌های پاریتی Node هم‌ارزی دو طرف را تضمین می‌کنند. نتیجهٔ عملی این رویکرد، رفتار یکسان دو نسخه روی همان تصویر است: خوانش، فیلدها، هشدارها و نگهبان ایمنی در هر دو یکسان‌اند و کاربر می‌تواند بسته به موقعیت از هر کدام استفاده کند.

در نسخهٔ **۲.۹.۰** دو محور تکمیل شده است: نخست، **توسعهٔ کیفیت پایپ‌لاین OpenCV** در پاسخ به بازخورد «خروجی مرتبط با OpenCV کیفیت لازم را ندارد» — افزودن حذف سایه و یکنواخت‌سازی نور، واریانت‌های جدید shadow_free و sharpened، دودویی مقیاس‌پذیر و صاف‌کردن چرخش دقیق‌تر. دوم، **بازسازی مستندات** با بخش اختصاصی «ساختار شکست کار (WBS) و نقشهٔ درختی مخزن» که در همین سند بخش ۵ آن آمده است.

■ بخش ۲

معماری دو زیرسیستمی + پل OCR

🖥️ زیرسیستم دسکتاپ — پایتون

Streamlit RTL (app.py) روی هستهٔ `label_reader/`: اعتبارسنجی → کیفیت → پیش‌پردازش OCR → OpenCV چهارموتوره → استخراج فیلد → پاسخ ایمن → TTS. موتورها: Tesseract 5 (پیش‌فرض)، Surya، EasyOCR و Qwen3-VL-2B-Instruct — به انتخاب کاربر از سایدبار. نسخهٔ دسکتاپ از v2.6.0 بخش «🔬 دیدن کار OpenCV و خوانش خام OCR» با دکمهٔ نمایش تصویر هر گام دارد.

📱 زیرسیستم وب/موبایل — جاوااسکریپت خالص

PWA تک‌صفحه‌ای (HTML/CSS/JS) بدون فریم‌ورک) با دوربین زندهٔ getUserMedia، OCR داخل مرورگر با Tesseract.js 6 روی WASM و دادهٔ زبان fas+eng تعبیه‌شده، پردازش تصویر با Canvas API (پورت عین قواعد OpenCV) و پخش صوتی با Web Speech API. بستهٔ اندروید (APK) همان PWA داخل WebView با پل بومی TextToSpeech است. نسخهٔ تک‌فایلی بدون پایتون هم دارد و از v2.7.0 همان بخش گام‌به‌گام OpenCV را نشان می‌دهد.

🛰️ پل (v2.5.0) OCR — پیوند دو زیرسیستم

سرور محلی استاندارد-کتابخانه‌ای (`label_reader/bridge.py`) که موتورهای سنگین دسکتاپ را برای رابط وب/موبایل قابل انتخاب می‌کند: `GET /ping` فهرست موتورهای آمادهٔ رایانه و `POST /ocr?engine=…` خوانش با موتور انتخابی — بدون ذخیره‌سازی و بدون لاگ محتوا. در انتخاب‌گر وب/موبایل گروه «موتورهای سنگین — پل دسکتاپ» با کادر آدرس و دکمهٔ بررسی اتصال آمده است.

اصل هم‌منطقی: خروجی هر قاعده در دو زیرسیستم باید یکسان باشد؛ به همین دلیل هر تغییر قاعده (مانند افزودن حذف سایه در v2.9.0) بلافاصله در هر دو زبان پیاده و با آزمون پاریتی تثبیت می‌شود. جزئیات پیاده‌سازی هر طرف در ستون «نگاشت درختی» جدول WBS (بخش ۵) با علامت ⇄ مشخص شده است.

■ بخش ۳

ملزومات طرح‌شده توسط استاد (استاد قربانی) و محل تحقق

الزامات پروژه در دو دسته نقل شده و برای هر الزام، محل تحقق، فایل پیاده‌سازی و آزمون ردیابی‌پذیر مشخص است. جدول انطباق کامل در رابط کاربری هر دو سامانه (بخش «📌 ملزومات طرح‌شده توسط استاد») و در بخش «ملزومات استاد» سند HTML نیز هست؛ خلاصهٔ آن در ادامه می‌آید.

الزام (دستهٔ ۱: تعریف مسئله)	محل تحقق	آزمون
«کاربر دوربین را به سمت جعبهٔ دارو یا بسته‌بندی محصول می‌گیرد»	دوربین زندهٔ getUserMedia (js/camera.js) + ورودی فایل هر دو رابط	e2e_v250.js (دوربین ساختگی)
«نام، اطلاعات چاپ‌شده و سایر فیلدهای قابل خواندن را استخراج می‌کند»	خط پردازش کامل (pipeline.py ⇄ js/pipeline.js) + */extraction	pytest + ۶۸ ۱۵۲ Node
«آن‌ها را به زبان فارسی می‌خواند»	پاسخ قالبی فارسی (response/builder.py) + TTS (Web Speech / edge-tts / پل بومی)	test_tts_llm.py + E2E

الزام (دستهٔ ۱: تعریف مسئله)	محل تحقق	آزمون
«اطلاعات خوانده‌شده را از توصیه‌های پزشکی جدا نگه می‌دارد»	نگهبان ایمنی (response/guard.py ⇄ js/guard.js) — ممنوع‌واژه‌ها و نظارت اعداد	test_security.py + آزمون پاریتی

الزام (دستهٔ ۲: نمودار معماری — ۵ مرحله)	فایل دسکتاپ	فایل موبایل
۱. OpenCV (تشخیص ناحیهٔ برچسب، اصلاح زاویه و بهبود وضوح)	preprocess.py + quality.py + validation.py	js/boxdetect.js + js/preprocess.js
۲. OCR (استخراج نام، اعداد و متن بسته‌بندی)	ocr/tesseract_provider.py + easyocr_provider.py (+ surya + vlm)	js/ocr.js (Tesseract.js/WASM)
۳. پردازش متن و کنترل فیلدها (نرمال‌سازی نام، استخراج قدرت و تعداد درج‌شده)	textnorm.py + extraction/patterns.py + fields.py	js/textnorm.js + patterns.js + extractor.js
۴. تولید پاسخ فارسی (منطق استخراج یا LLM بازنویسی وفادار)	response/builder.py + guard.py (+ llm/rewriter.py اختیاری)	js/responder.js + guard.js
۵. TTS + Streamlit (خواندن اطلاعات و نمایش متن استخراج‌شده)	tts/* + response/tts_prep.py + app.py	js/app.js (speechSynthesis) + index.html

هر یک از این پنج مرحله در حین پردازش واقعی نیز به‌صورت زنده اعلام می‌شود: پیام «⏳ مرحلهٔ n در حال اجرا» و سپس «✅ انجام شد — این بخش از نیازمندی استاد برآورده شد» همراه با خروجی واقعی و معیارهای ارزیابی همان مرحله (تیزی/کنتراست تصویر، اطمینان OCR، پوشش شواهد، عبور از نگهبان، وضعیت صوت) در خط‌زمان رنگی هر دو رابط نمایش داده می‌شود و گزارش JSON ارزیابی مرحله‌ای نیز قابل دانلود است. این اعلام از v1.6.0 در هر دو زیرسیستم فعال است (stages.py + stage_callback ⇄ onStage در js/pipeline.js).

■ بخش ۴

چگونه کار می‌کند؟ — خط پردازش ۹ مرحله‌ای

این خط در هر دو زیرسیستم با همان ترتیب اجرا می‌شود؛ تفاوت فقط در پیاده‌سازی است (OpenCV ⇄ Canvas)، نه در قواعد. تکمیل هر مرحله هنگام پردازش به کاربر اعلام و خروجی و ارزیابی کیفیت همان مرحله نمایش داده می‌شود.

۱) ورود و اعتبارسنجی تصویر

بررسی امضای باینری (magic bytes) PNG/JPEG/WebP، سقف حجم ۱۰ مگابایت و حداقل ابعاد — فایل‌های غیرتصویری و مخرب رد می‌شوند تا موتور OCR با ورودی ناایمن روبه‌رو نشود.

۲) ارزیابی کیفیت

تاری (واریانس لاپلاسین)، تاریکی (میانگین روشنایی) و انعکاس نور (نسبت پیکسل‌های خیلی روشن با هیوریستیک پس‌زمینه‌آگاه) سنجیده می‌شوند؛ هشدارها به‌صورت چیپ رنگی اعلام می‌شوند تا کاربر در صورت نیاز عکس بهتری بگیرد.

۳) پیش‌پردازش (OpenCV / Canvas) — توسعه‌یافته در v2.9.0

مقیاس هوشمند کوتاه‌ترین ضلع تا ~۱۰۰۰ پیکسل با درون‌یابی لانسزوس؛ **حذف سایه و یکنواخت‌سازی نور** با تقسیم تصویر بر پس‌زمینهٔ برآوردی (morphological close بیضوی ≈ ۱/۸ ضلع)؛ CLAHE برای تقویت کنتراست موضعی؛ **تیزکردن Unsharp Mask** برای لبه‌های نرم‌شدهٔ عکس موبایل؛ صاف‌کردن چرخش کم‌زاویه تا ±۱۵ درجه با روش دومرحله‌ای؛ و ساخت **۷ نسخهٔ پردازشی**: اصلی، خاکستری+CLAHE، حذف سایه (shadow_free)، تیزشده (sharpened)، دودویی، متن روشن روی تیره و جداسازی جوهر رنگی. نسخه‌های مکمل فقط وقتی خوانش نسخه‌های اصلی ضعیف است (اطمینان زیر ۰٫۸) به OCR داده می‌شوند.

۴) تشخیص و برش جعبهٔ دارو (موبایل)

ناحیهٔ بسته با لبه‌یابی سوبل، آستانهٔ درصدی، اتساع و بزرگ‌ترین جزء همبند پیدا و از پس‌زمینه برش داده می‌شود — نویز OCR به‌شدت کم می‌شود؛ در دسکتاپ همین نقش را برش دستی کاربر ایفا می‌کند.

۵) OCR چندنسخه‌ای و چند-PSM

هر نسخهٔ تصویر با حالت‌های چیدمان ۳ و ۶ (و ۱۱ پراکنده در موبایل) خوانده می‌شود؛ user_defined_dpi تنظیم شده و گذرهای زبانی fas و eng جدا اجرا می‌شوند (ترکیب fas+eng ریشهٔ تزریق حروف لاتین به فارسی بود و حذف شد).

۶) ادغام فازی و رأی اجماعی

خوانش‌های مشابه (شباهت ترتیبی ≤ ۰٫۸) ادغام می‌شوند؛ گیت کیفیت قطعه‌ها زبالهٔ ترکیب‌خطی، نماد محض و کادر منحط را حذف می‌کند؛ ادغام «بهترین-اول» بر اساس امتیاز واریانت انجام و رأی اجماعی قطعه‌های تک‌رأی کم‌اطمینان را می‌ریزد. در خوانش ضعیف، چرخش‌های ۹۰ درجه خودکار آزموده می‌شود.

۷) استخراج فیلدها و تناقض‌یابی

نام، مادهٔ مؤثره، قدرت درج‌شده، شکل دارویی، تعداد/حجم/وزن، تاریخ انقضا، سری و سازنده با الگوهای دو زبانه استخراج و هر مقدار به شناسهٔ قطعه‌های شاهد گره می‌خورد؛ خوانش‌های متناقض به‌جای انتخاب خاموش، صریح اعلام می‌شوند (محافظ مقدار صفر فعال است).

۸) تولید پاسخ فارسی

مقدمه بر اساس نوع بسته (دارو/محصول عمومی) انتخاب و پاسخ از قالب‌های فارسی ساخته می‌شود — بدون هیچ مدل زبانی ابری؛ مسیر LLM اختیاری است و اگر فعال شود، خروجی‌اش باز هم از نگهبان عبور می‌کند.

۹) نگهبان ایمنی و خروجی

واژه‌های ممنوع (مانند «دوز مصرف») رد، اعداد خارج از شواهد مسدود و سلب مسئولیت تزریق می‌شود؛ سپس نمایش، پخش صوتی (فقط با رضایت صریح) و دانلود JSON انجام می‌گیرد.

■ بخش ۵ — جدید در v2.9.0

ساختار شکست کار (WBS) و نقشهٔ درختی مخزن

پروژه با روش «ساختار شکست کار» (Work Breakdown Structure) مدیریت می‌شود: کل کار به دوازده بستهٔ کاری (WBS-1 تا WBS-12) شکسته شده و هر بستهٔ کاری دقیقاً به پوشه‌ها و فایل‌های همان ساختار درختی نگاشت می‌شود — یعنی جای هر فایل در مخزن، حکم شمارهٔ آن در طرح شکست کار است. به این ترتیب «ساختار شکست کار» و «ساختار درختی پوشه‌ها» دو نمایِ یکی هستند: نمای منطقی (چه کاری) و نمای فیزیکی (کدام فایل). جدول زیر نگاشت هر بستهٔ کاری را نشان می‌دهد و درخت کامل مخزن با کد WBS هر شاخه در ادامه آمده است.

کد	بستهٔ کاری	خروجی و تحویل‌شدنی	پوشه‌ها / فایل‌ها (نگاشت درختی)
WBS-1	مدیریت پروژه، نسخه‌گذاری و انتشار	README، شمارهٔ نسخهٔ واحد، اسکریپت‌های ساخت ZIP/تک‌فایل/Release، لانچرهای شروع	`README.md` · `VERSION.txt` · `scripts/build_*.py` · `scripts/make_release_*.py` · `mobile-app/START-*` ·
WBS-2	تحلیل نیازمندی‌ها (ملزومات استاد)	نگاشت الزام‌به‌فایل، اعلام مرحله‌به‌مرحلهٔ پردازش، جدول انطباق	`label_reader/stages.py` · بخش «ملزومات استاد» در رابط‌ها و سندها
WBS-3	معماری و مدل داده	سند معماری، مدل‌های Pydantic با ردیابی شواهد، پیکربندی مرکزی	`docs/ARCHITECTURE.md` · `models.py` · `config.py`
WBS-4	اعتبارسنجی و پیش‌پردازش تصویر (OpenCV)	magic bytes/سقف حجم، سنجش تاری/نور/انعکاس، مقیاس لانسزوس، حذف سایه، CLAHE، تیزکردن، deskew، ۷ نسخهٔ پردازشی + مسیر گام‌به‌گام	`validation.py` · `quality.py` · `preprocess.py` ⇄ `js/boxdetect.js` + `js/preprocess.js`
WBS-5	موتورهای OCR (چهار موتور + پل مرورگر)	Tesseract (پیش‌فرض)، EasyOCR، Surya، Qwen3-VL-2B-Instruct — همه از گیت کیفیت و رأی اجماعی عبور می‌کنند	`label_reader/ocr/*` ⇄ `js/ocr.js` · `models/tessdata/` · `mobile-app/vendor/ + lang/`
WBS-6	پردازش متن، استخراج فیلد و پاسخ ایمن	نرمال‌سازی نویسه، الگوهای دو زبانه، وضعیت پنج‌گانهٔ فیلد، تناقض‌یابی، نگهبان ایمنی، LLM اختیاری وفادار	`textnorm.py` · `extraction/*` · `response/*` · `llm/*` ⇄ `js/textnorm · patterns · extractor · responder · guard`
WBS-7	رابط دسکتاپ (Streamlit)	سایدبار RTL، برش دستی، خط‌زمان ۵ مرحله، بخش «🔬 دیدن کار OpenCV»، مودال‌های راهنما، JSON	`app.py` · `ui_helpers.py` · `ui_theme.py` · `help_content.py` · `.streamlit/config.toml`
WBS-8	زیرسیستم وب/موبایل (PWA)	رابط تک‌صفحه‌ای RTL، دوربین زنده، TTS، PWA نصب‌شدنی آفلاین، نسخهٔ تک‌فایلی بدون پایتون	`index.html` · `css/` · `js/app · camera · pipeline · help` · `manifest.webmanifest` · `sw.js` · `label-reader-singlefile.html`

کد	بستهٔ کاری	خروجی و تحویل‌شدنی	پوشه‌ها / فایل‌ها (نگاشت درختی)
WBS-9	بستهٔ اندروید (APK)	WebView با دوربین و پل TTS بومی، ساخت بدون Gradle با aapt2/d8/apksigner	`android/MainActivity.java` · `AndroidManifest.xml` · `build_apk.sh` · `README-ANDROID-fa.md`
WBS-10	خوانش صوتی فارسی (TTS)	edge-tts (عصبی) با جایگزین gTTS — فقط با رضایت صریح کاربر	`label_reader/tts/*` · `response/tts_prep.py`
WBS-11	آزمون و تضمین کیفیت	pytest + ۶۸ Node + ۱۸ E2E ۱۵۲ مرورگر، ۱۴ نمونهٔ برچسب، ارزیابی دقت فیلد	`tests/` · `tests-js/` · `sample_images/` · `docs/evaluation/` · `scripts/evaluate.py`
WBS-12	مستندسازی	سند HTML + همین PDF، راهنمای نصب، محدودیت‌ها، تاریخچهٔ نسخه‌ها	`docs/*.md` · `docs/*.pdf` · `mobile-app/documentation.html`

چطور این جدول را بخوانیم؟ ستون «نگاشت درختی» همان مسیر فایل در مخزن است؛ علامت ⇄ یعنی همان بستهٔ کاری در دو زیرسیستم دسکتاپ (پایتون) و وب/موبایل (جاوااسکریپت) با منطق عیناً یکسان پیاده‌سازی شده و آزمون‌های پاریتی `tests-js/` همین هم‌ارزی را تضمین می‌کنند. سه قاعدهٔ حاکم بر ساختار: ۱) هر زیرسیستم منطقی یک پوشهٔ اختصاصی دارد و مرزها با IIFE / `__init__.py` مشخص است؛ ۲) هر قاعدهٔ پردازشی دقیقاً دو بار (پایتون و جاوااسکریپت) با آزمون پاریتی پیاده شده تا خروجی دسکتاپ و موبایل یکی باشد؛ ۳) داده‌های حجیم و غیرکد (مدل، فونت، وندور) از کد تفکیک شده‌اند تا مخزن و ZIP قابل حمل سبک بمانند.

نقشهٔ درختی زیر همان نگاشت را به‌صورت فیزیکی نشان می‌دهد؛ کد بستهٔ کاری هر شاخه داخل کروشه درج شده است تا مسیر «الزام ← بستهٔ کاری ← فایل» در یک نگاه ردیابی‌پذیر باشد:

```
LabelVoice/
├ README.md · VERSION.txt · requirements*.txt        [WBS-1]
├ app.py                                             [WBS-7]  رابط Streamlit دسکتاپ
├ label_reader/                                      ← هستهٔ پایتون
|   ├ config.py · models.py                          [WBS-3]  پیکربندی و مدل داده
|   ├ stages.py                                      [WBS-2]  لام ۵ مرحله + ارزیابی هر مرحله
|   ├ validation.py · quality.py                     [WBS-4]  اعتبارسنجی ورودی و سنجش کیفیت
|   ├ preprocess.py                                  [WBS-4]  OpenCV: لانزوس، حذف سایه، CLA
|   |                                                        تیزکردن، deskew، ۷ نسخه + ope
|   ├ ocr/                                           [WBS-5]  tesseract/easyocr/surya/vlm p
|   ├ textnorm.py · extraction/                      [WBS-6]  ال‌سازی + استخراج فیلد و تناقض
|   ├ response/ · llm/                               [WBS-6]  پاسخ فارسی + نگهبان ایمنی
|   ├ tts/                                           [WBS-10] edge-tts / gTTS
|   └ ui_helpers.py · ui_theme.py · help_content.py  [WBS-7]  اجزای رابط و راهنما
├ mobile-app/                                        ← زیرسیستم وب/موبایل
|   ├ index.html · css/ · manifest.webmanifest       [WBS-8]  پوستهٔ PWA
```

```
│   ├─ js/  (textnorm · patterns · extractor ·        [WBS-5,6] پورت عین قواعد پایتون
│   │        responder · guard · pipeline ·
│   │        preprocess · boxdetect · ocr · camera)
│   ├─ sw.js · vendor/ · lang/                        [WBS-8]  آفلاین کامل (کش + WASM + زبان
│   ├─ START-WINDOWS.bat · server-windows.ps1 · …     [WBS-1]  لانچرهای بدون پایتون
│   ├─ label-reader-singlefile.html                   [WBS-8]  نسخهٔ تک‌فایلی
│   └─ documentation.html                             [WBS-12] سند فنی HTML
├─ android/                                           [WBS-9]  WebView + پل TTS بومی + build
├─ models/tessdata/                                   [WBS-5]  fas + eng (tessdata_best)
├─ sample_images/                                     [WBS-11] نمونهٔ برچسب آزمون ۱۴
├─ tests/                                             [WBS-11] pytest (آزمون ۱۵۲)
├─ tests-js/                                          [WBS-11] Node + E2E (آزمون ۸۲)
├─ docs/                                              [WBS-12] PDFها ،معماری، نصب، محدودیت‌ها
└─ scripts/                                           [WBS-1,11] ساخت/انتشار/ارزیابی/اثبات‌ها
```

جزئیات نقش تک‌تک فایل‌ها (به تفکیک دسته و با توضیح هر فایل) در بخش ۶ آمده است؛ بخش ۵ چارچوب کلان را می‌دهد و بخش ۶ مرجع جزییات است.

■ بخش ۶

فایل‌های سامانه — مرجع فشرده

جدول زیر مهم‌ترین فایل‌های هر دسته را با نقششان فهرست می‌کند؛ فهرست کامل با توضیح تک‌تک فایل‌ها در سند HTML (بخش «فایل‌های سامانه») و درخت بالا موجود است.

فایل	نقش
`app.py`	رابط Streamlit RTL: ورودی فایل/دوربین، برش دستی، خط‌زمان مراحل، بخش «🔬 دیدن کار OpenCV» با دکمهٔ نمایش تصویر هر گام، اصلاح دستی، صوت، JSON
`label_reader/pipeline.py`	هماهنگ‌کنندهٔ کل خط پردازش + انتشار opencv_trace و variant_readings
`label_reader/preprocess.py`	OpenCV: مقیاس لانسزوس، حذف سایه و یکنواخت‌سازی نور (v2.9.0)، CLAHE، تیزکردن، deskew پروفایلی، ۷ نسخهٔ خروجی + build_variants_traced
`label_reader/ocr/*`	چهار موتور خوانش: tesseract (دو-سه PSM + گذر زبانی جدا)، easyocr (مکمل)، surya (اختیاری)، vlm (Qwen3-VL-2B-Instruct)
`label_reader/extraction/*` + `response/*`	الگوهای دو زبانه، وضعیت پنج‌گانهٔ فیلد، تناقض‌یابی، قالب پاسخ فارسی و نگهبان ایمنی
`label_reader/stages.py`	اعلام ۵ مرحلهٔ ملزومات استاد + ارزیابی کیفیت هر مرحله + گزارش JSON

فایل	نقش
`mobile-app/js/preprocess.js`	پورت مرورگری: مقیاس، خاکستری، یکنواخت‌سازی نور (v2.9.0)، Unsharp، دودویی، deskew + مسیر گام‌به‌گام
`mobile-app/js/ocr.js`	پوشش Tesseract.js: نسخه‌ها، چند PSM، ادغام فازی، گزارش پیشرفت
`mobile-app/js/boxdetect.js` + `camera.js`	تشخیص جعبهٔ دارو (سوبل + جزء همبند) و دوربین زنده (۳ فریم، انتخاب تیزترین)
`mobile-app/sw.js` + `manifest.webmanifest`	سرویس‌ورکر cache-first برای آفلاین کامل + مانیفست نصب PWA
`android/MainActivity.java` + `build_apk.sh`	WebView با مجوز دوربین + پل بومی TextToSpeech + ساخت APK بدون Gradle
`tests/` + `tests-js/`	۱۵۲ آزمون pytest + ۶۸ آزمون E2E + ۱۸ Node مرورگر (پاریتی و امنیت)
`scripts/`	ساخت ZIP و تک‌فایل، انتشار Release، ارزیابی (evaluate.py) و اثبات‌های کیفی
`docs/` + `mobile-app/documentation.html`	سند معماری، نصب، محدودیت‌ها، نسخه‌های PDF و همین سند HTML

■ بخش ۷

کتابخانه‌ها، ابزارها و فناوری‌ها

همهٔ فناوری‌های به‌کاررفته با نقش دقیق‌شان:

Python 3.12 — هستهٔ دسکتاپ و اسکریپت‌ها

OpenCV — پیش‌پردازش تصویر (v2.9.0: حذف سایه + Unsharp)

Tesseract 5 + pytesseract — OCR پیش‌فرض (LSTM)

EasyOCR (PyTorch) — موتور دوم مکمل

Surya OCR — موتور سوم اختیاری

Qwen3-VL-2B-Instruct — موتور چهارم + دستیار بینایی

Pydantic v2 — مدل داده با ردیابی شواهد

Streamlit — رابط دسکتاپ RTL

Pillow — تبدیل تصویر + رندر نمونه‌ها (raqm)

edge-tts / gTTS — خوانش صوتی فارسی

Tesseract.js 6 (WASM) — OCR مرورگر

Canvas API — پردازش پیکسلی موبایل

getUserMedia — دوربین زنده

Web Speech API — پخش صوتی مرورگر

PWA + Service Worker — نصب و آفلاین

Android WebView — پوستهٔ APK

Vazirmatn — فونت فارسی متن‌باز

۱۵۲ — pytest آزمون دسکتاپ

۶۸ — Node.js آزمون واحد + E2E

📱 **اپ موبایل با چه رویکردی توسعه داده شده؟** رویکرد اصلی PWA است: اپ وب تک‌صفحه‌ای با HTML/CSS/JavaScript خالص (بدون فریم‌ورک) که با Web App Manifest روی اندروید/iOS نصب می‌شود و با Service Worker (کش cache-first) کاملاً آفلاین اجرا می‌شود. OCR داخل مرورگر با Tesseract.js 6 روی WebAssembly و دادهٔ زبان fas+eng تعبیه‌شده انجام می‌شود؛ دوربین زنده با استاندارد getUserMedia (انتخاب تیزترین فریم از ۳ فریم)؛ پردازش تصویر با Canvas API (پورت عین قواعد OpenCV دسکتاپ)؛ خوانش صوتی در مرورگر با Web Speech API و در APK با پل بومی TextToSpeech؛ بستهٔ اندروید همان PWA داخل Android WebView است. نسخهٔ تک‌فایلیِ بدون پایتون نیز با `scripts/build_singlefile.py` ساخته می‌شود.

■ بخش ۸

مدل‌ها و موتورها — شفافیت کامل

کاربر همیشه باید بداند برای هر بخش از چه مدل یا موتوری استفاده می‌شود — این جدول رسمی سامانه است و در رابط کاربری (سایدبار دسکتاپ + راهنمای وب) و حین مراحل پردازش نیز اعلام می‌شود. تک‌منبع حقیقت: `label_reader/models_info.py` ؛ نسخه‌ها در زمان اجرا از دستگاه کاربر خوانده می‌شوند.

بخش سامانه	مدل / موتور	نقش
پیش‌پردازش تصویر	**OpenCV** (کلاسیک، بدون مدل یادگیری)	CLAHE + حذف سایه + تیزکردن Unsharp + deskew + ۷ نسخهٔ پردازشی
OCR پیش‌فرض دسکتاپ	**Tesseract 5** (LSTM) + pytesseract	خوانش fas و eng جدا + PSM 3/6/11 + ادغام فازی
OCR مکمل دسکتاپ	**EasyOCR** (PyTorch)	پرکردن نواحی ضعیف تسراکت؛ بارگیری lazy
OCR اختیاری سوم	**Surya OCR** (≥0.17)	تشخیص خط + بازشناسی ترنسفورمری چندزبانه
OCR اختیاری چهارم	**Qwen/Qwen3-VL-2B-Instruct**	مدل زبانی-بینایی محلی؛ کل تصویر را یکجا می‌خواند؛ شناسهٔ شاهد vlm-*
OCR مرورگر	**Tesseract.js 6** (WASM LSTM)	fas/eng/fas+eng با دادهٔ integer و best؛ کاملاً محلی
دادهٔ زبان	**tessdata_best fas + eng**	مدل‌های دقیق Tesseract — محلی در models/tessdata و vendor
خوانش صوتی	**edge-tts** (fa-IR-FaridNeural) / **gTTS** / Web Speech / پل TextToSpeech اندروید	فقط با رضایت صریح کاربر

بخش سامانه	مدل / موتور	نقش
پیکربندی و مدل داده	pydantic-settings + Pydantic v2	تنظیمات محیطی LBR_* + مدل‌های شواهدمحور

■ بخش ۹

حریم خصوصی و خطوط ایمنی

ایمنی در این پروژه ویژگی افزودنی نیست؛ ستون معماری است و با آزمون‌های اختصاصی (`tests/test_security.py`) و نگهبان جاوااسکریپت (`js/guard.js`) محافظت می‌شود. چهار خط قرمز اصلی:

۱) جدایی اطلاعات از توصیه

سامانه فقط «بازخوانی متن روی بسته» است. واژه‌های ممنوع (دوز مصرف، تجویز، مصرف روزانه و...) در پاسخ رد می‌شوند؛ «قدرت درج‌شده» هرگز «دوز مصرف» نامیده نمی‌شود و هر پاسخ با سلب مسئولیت صریح همراه است.

۲) نظارت اعداد و ردیابی‌پذیری

هیچ عددی که در شواهد OCR نیامده باشد در پاسخ ظاهر نمی‌شود؛ هر فیلد استخراج‌شده به شناسهٔ قطعه‌های شاهد (*_ocr یا *-vlm) متصل است و پوشش شواهد در ارزیابی مرحلهٔ سوم اعلام می‌شود.

۳) پردازش صددرصد محلی

تصویر و متن هرگز به سروری ارسال نمی‌شود؛ OCR، پیش‌پردازش، استخراج، پاسخ و گزارش‌گیری همه روی خود دستگاه اجرا می‌شوند. در لاگ‌های پیش‌فرض هیچ متن یا تصویری ثبت نمی‌شود و پل OCR نیز محتوا را ذخیره یا لاگ نمی‌کند.

۴) رضایت صریح برای صوت

تولید و پخش صوت تنها با کلیک آگاهانهٔ کاربر روی دکمهٔ پخش انجام می‌شود؛ هیچ صوتی خودکار پخش نمی‌شود و متن پیش از خوانش فقط برای تلفظ (اعداد و علائم) آماده‌سازی می‌شود نه تغییر معنا.

■ بخش ۱۰

ارزیابی و آزمون — نتایج واقعی

موضوع این سامانه دارو است و هر اشتباه می‌تواند آسیب‌زا باشد؛ بنابراین ارزیابی بر اساس اصول سنجش سامانه‌های حساس انجام شده است: سنجش در سطح فیلد، شمردن غیبت و تعارض به‌عنوان خطا، و هیچ عدد جعلی. همهٔ اعداد از اجرای واقعی در محیط توسعه به‌دست آمده و با `scripts/evaluate.py` قابل بازتولید است.

سنجه	مقدار	توضیح
دقت استخراج فیلد (Tesseract)	**۱۲ از ۱۴ = ٪۸۵٫۷**	روی مجموعهٔ ۱۴ نمونهٔ استاندارد (فارسی/انگلیسی/دوزبانه/چرخیده)
پاریتی موبایل با دسکتاپ	**۱۲ از ۱۴ = ٪۸۵٫۷**	همان نتیجه با Tesseract.js — آزمون‌های پاریتی Node
میانگین زمان خوانش دسکتاپ	**~۱٫۹ ثانیه**	Tesseract روی ۷ نسخه با گیت کیفیت
میانگین زمان خوانش مرورگر	**۰٫۸ تا ۲٫۷ ثانیه**	بسته به دستگاه و کیفیت دادهٔ زبان
آزمون‌های خودکار	**۲۳۸ پاس**	pytest + ۶۸ Node + ۱۸ E2E ۱۵۲ مرورگر

اثبات کیفیت OpenCV در v2.9.0: روی نمونهٔ ساختگی «عکس موبایل با سایهٔ سنگین ۵۸٪»، خوانش خام نسخهٔ original عملاً آشغال بود (۹۰ قطعهٔ تک‌حرفی، اطمینان ۰٫۲۳، صفر برخورد کلیدواژه)؛ پس از افزودن حذف سایه، واریانت shadow_free و gray_clahe بهبودیافته متن را درست خواندند («استامینوفن… کپسول‌های نرم گوارشی… تاریخ انقضا: ۱۴۰۶/۰۵») با اطمینان تا ۰٫۷۴ و ۳ برخورد کلیدواژه — شواهد تصویری و متنی در `scripts/evidence_v290/` مخزن و اسکریپت بازتولید `scripts/ab_opencv_v290.py`. محدودیت شناخته‌شده: ارقام فارسی در هر دو موتور OCR ضعیف خوانده می‌شوند و با محافظ «مقدار نامطمئن + اصلاح دستی» مدیریت می‌شوند.

■ بخش ۱۱

محدودیت‌های شناخته‌شده (شفافیت کامل)

۰۹ — ارقام فارسی در OCR

هر دو موتور (Tesseract و Tesseract.js) ارقام فارسی را ضعیف می‌خوانند (مثلاً ۵۰۰ → ۰). محافظ «مقدار نامطمئن» این موارد را علامت می‌زند و اصلاح دستی دارد؛ ورود مقدار ساختگی به پاسخ مسدود است.

موتورهای سنگین فقط دسکتاپ

EasyOCR و Surya و Qwen3-VL-2B-Instruct در مرورگر اجرا نمی‌شوند (نیازمند پایتون/مدل‌اند)؛ در وب از طریق پل OCR دسکتاپ قابل انتخاب‌اند و در انتخاب‌گر هم همین محدودیت شفاف اعلام می‌شود.

بارکدخوان وجود ندارد

خواندن شمارهٔ بارکد به‌تنهایی شناسایی معتبر محصول نیست؛ در این نسخه بارکدخوان پیاده نشده و شناسایی محصول صرفاً از متن چاپ‌شده انجام می‌شود.

طبقه‌بندی نوع بسته تأیید هویت نیست

تشخیص «دارو / محصول عمومی» فقط برای انتخاب قالب فیلدهاست و به هیچ وجه تأیید اصالت یا هویت دارو محسوب نمی‌شود.

تصاویر چندمحصولی

اطلاعات چند بسته در یک خروجی ادغام نمی‌شود؛ کاربر باید یک بسته را برش بزند. برش دستی در دسکتاپ و تشخیص خودکار جعبه در موبایل این را تسهیل می‌کنند.

■ بخش ۱۲

نصب و اجرا

📦 بستهٔ قابل حمل (بدون پایتون)

از Releases فایل **Portable-Web.zip** را بگیرید، باز کنید و `START-WINDOWS.bat` را اجرا کنید — صفحهٔ شروع خودترمیم (START-HERE) باز می‌شود؛ لانچر نسخه‌محور است و همیشه صفحهٔ شروعِ هم‌نسخهٔ خودش را باز می‌کند. در لینوکس/مک: `شروع-در-لینوکس-مک.sh` .

🖥️ نسخهٔ دسکتاپ (پایتون)

`pip install -r requirements.txt` سپس `streamlit run app.py` . موتورهای اختیاری: `requirements-surya.txt` / `requirements-easyocr.txt` ؛ دادهٔ زبان در `models/tessdata` آماده است (`scripts/download_tessdata.py`).

📱 نصب روی موبایل (PWA)

سایت را در Chrome/Edge اندروید باز کنید و «Add to Home Screen» بزنید — پس از بار اول کاملاً آفلاین اجرا می‌شود (سرویس‌ورکر cache-first).

🤖 نسخهٔ APK اندروید

فایل **Android.apk** از Releases را نصب کنید (نسخهٔ فعلی: ۲.۹.۰ / versionCode 20). اگر نسخهٔ قدیمی‌تری نصب دارید و نصب نشد، ابتدا Uninstall کنید (کلید امضا ممکن است متفاوت باشد).

📄 تک‌فایل بدون سرور

`label-reader-singlefile.html` را دابل‌کلیک کنید — موتور OCR آفلاین (WASM + دادهٔ زبان) داخل فایل تعبیه شده و حتی با file:// بدون اینترنت کار می‌کند.

■ بخش ۱۳

تاریخچه نسخه‌ها

توسعهٔ کیفیت پایپ‌لاین OpenCV + WBS و درخت مخزن در مستندات v2.9.0

در پاسخ به «خروجی OpenCV کیفیت لازم را ندارد»: تابع `flatten_illumination` (حذف سایه با تقسیم بر پس‌زمینهٔ برآوردی)، واریانت‌های جدید shadow_free و sharpened، تزریق حذف سایه به gray_clahe و binarized، بلوک آستانه‌گذاری مقیاس‌پذیر (۳۱..۹۱) + medianBlur، بزرگ‌نمایی INTER_LANCZOS4 و صاف‌کردن چرخش دومرحله‌ای (minAreaRect + پالایش پروفایلی ±۲°)؛ گیت هوشمند اجرای واریانت‌های مکمل فقط در خوانش ضعیف؛ اثبات واقعی روی نمونهٔ سایه‌دار ۵۸٪ (خوانش صحیح استامینوفن/تاریخ انقضا با اطمینان ۰٫۷۴ در برابر خوانش آشغال original) — و در مستندات: بخش «ساختار شکست کار (WBS) و نقشهٔ درختی مخزن» با ۱۲ بستهٔ کاری و نگاشت درخت فایل‌ها، در هر دو سند HTML و همین Node ۴ + pytest ۱۴. PDF جدید (مجموع ۲۳۸ آزمون).

دکمهٔ «👁 نمایش تصویر پس از این عملیات» برای هر گام + لانچر نسخه‌محور v2.8.0

به‌ازای هر گام مسیر OpenCV در دسکتاپ یک دکمهٔ نمایش تصویر خروجی همان گام (با کلید «نمایش همهٔ تصاویر یک‌جا») و در وب/موبایل همان الگوی دکمه‌ای روی تایم‌لاین؛ لانچر START-WINDOWS نسخه‌محور شد.

«دیدن کار OpenCV و خوانش خام OCR» در وب/موبایل v2.7.0

بخش شفافیت گام‌به‌گام (تصویر پس از هر عملیات + خوانش خام هر نسخه قبل از ادغام) در وب تک‌فایلی/PWA/APK؛ کش v19 → SW.

دیدن گام‌به‌گام کار OpenCV و خوانش خام OCR در دسکتاپ v2.6.0

بخش «🔬 دیدن کار OpenCV و خوانش خام OCR (گام‌به‌گام)» در Streamlit: تصویر خروجی هر عملیات با عین پارامترهای cv2 + معیارهای هر گام + خوانش خام هر واریانت با کادر و جدول قطعه‌ها.

انتخاب ۹موتوره در وب/موبایل با «پل OCR» + رفع پخش صوتی اندروید + هدر پالتی v2.5.0

پل OCR دسکتاپ برای موتورهای سنگین وب/موبایل؛ پل بومی TextToSpeech برای رفع بی‌صدایی WebView؛ هدر با پالت سفید/آبی/سبز/قرمز به انتخاب کاربر.

انتخاب‌گر موتور/مدل خوانش در وب و موبایل + هدر تصویری v2.4.0

انتخاب‌گر همیشه‌نمایان ۵ گزینهٔ مرورگری با ذخیرهٔ انتخاب + رفع «START-WINDOWS نسخهٔ قدیم را اجرا می‌کند» + هدر تصویری اختصاصی در هر سه رابط.

موتور چهارم قابل انتخاب Qwen3-VL-2B-Instruct v2.3.0

منوی ۵گزینه‌ای دسکتاپ: Tesseract (پیش‌فرض) · Qwen3-VL-2B-Instruct · Surya · EasyOCR · خودکار — همه از گیت کیفیت و رأی اجماعی عبور می‌کنند.

موتور سوم اختیاری Surya + پالت دارویی پررنگ‌تر v2.2.0

Surya OCR به انتخاب کاربر + پس‌زمینهٔ چندلایهٔ آبی/سبز/قرمز روی سفید + ریبون چهاررنگ مشترک.

لانچر خودترمیم + رجیستری مدل‌ها + دستیار بینایی v2.0.0

رفع ریشه‌ای «start.bat سایت به‌هم‌ریخته» (سرویس‌ورکر قدیمی) با صفحهٔ شروع خودترمیم؛ رجیستری مرکزی models_info.py با نسخهٔ زندهٔ مدل‌ها؛ دستیار اختیاری Qwen3-VL.

بازطراحی کامل گرافیک وب با سیستم طراحی توکن‌محور v1.9.0

بازنویسی کل CSS با پالت برند، مقیاس‌های استاندارد و رفع باگ حالت تاریک — پایان ناهماهنگی درشت‌نمایی و سرریز.

بازطراحی موتور خوانش فارسی + گرافیک جدید v1.8.0

گذرهای زبانی جدا، جداسازی جوهر رنگی، رأی اجماعی، موتور دوم مکمل، دروازه‌های نام سخت‌گیرانه‌تر — نتیجه روی جعبهٔ واقعی لانزول صحیح شد.

اصلاح اساسی کیفیت OCR (متن سفید روی زمینهٔ رنگی) v1.7.0

واریانت white_on_dark، ادغام بهترین-اول، گیت کیفیت قطعه‌ها، دروازهٔ نام معتبر.

اعلام مرحله‌به‌مرحله پردازش + ارزیابی هر مرحله v1.6.0

اعلام «الان کدام بخش از نیازمندی استاد انجام شد» + خروجی و معیارهای ارزیابی هر مرحله + گزارش JSON.

بخش «ملزومات استاد» با جدول انطباق v1.5.0

نقل عین ملزومات در دو دسته + جدول انطباق الزام ← تحقق ← فایل ← آزمون در هر دو رابط و هر دو سند.

نام توسعه‌دهندگان + بخش ارزیابی مدل‌ها v1.4.0

نام «حمید جلیلوند و نوید بدری» + بخش ارزیابی با نمودارهای واقعی + نگاشت فایل‌ها.

دوربین زنده، تشخیص جعبه، کیفیت بالاتر OCR v1.2.0

getUserMedia با انتخاب تیزترین فریم، PSM 11، boxdetect، تلاش چرخش ۹۰° و پاسخ غنی‌تر.

مودال‌های راهنما + مستندات مصور + PDF v1.3.0

راهنمای مودالی فناوری در هر دو رابط + سند فنی HTML رسپانسیو + PDF شکیل.

انتشار عمومی اول v1.0.0

دسکتاپ کامل + PWA آفلاین + آزمون‌های خودکار + GitHub Pages و Releases.

💊 برچسب‌خوان هوشمند دارو و محصولات

سامانه فقط متن روی بسته را می‌خواند — نه بیش و نه کم. قدرت درج‌شده «دوز مصرف» نیست، هیچ توصیهٔ پزشکی تولید نمی‌شود و هر فیلد به شواهد OCR متصل است. پردازش صددرصد روی دستگاه شما انجام می‌شود.

🌐 مخزن: github.com/hjalilvand2004-coder/LabelVoice

📦 دانلود نسخه‌ها: Releases · 📄 سند HTML: documentation.html

👷 توسعه و طراحی: حمید جلیلوند و نوید بدری — نسخه ۲.۹.۰ · اکتبر ۲۰۲۶